

Comp 2140: Computer Languages, Grammars, and Translators (Compiler)

Prof. Jianguo Lu

January 6, 2026

Why we are here

One picture for Comp2540

before

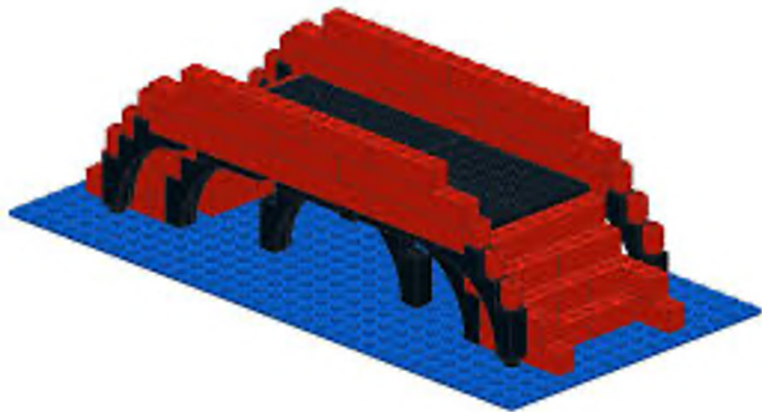


after



One picture for Comp2140

before



before



after



Gordie Howe bridge being constructed in Windsor

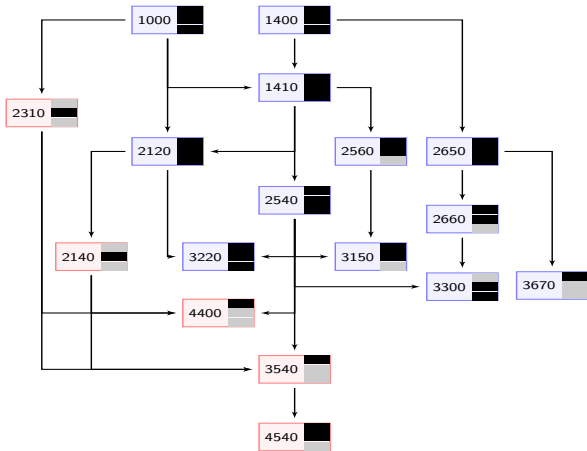
Why we are here



- ▶ In the past, we learnt how to write **small** programs
- ▶ After this course, we can write **large** programs
- ▶ How large is 'large'?
 - ▶ Thousands or tens of thousands of lines
- ▶ We don't write large programs manually
 - ▶ We generate them automatically
- ▶ How do we know the program is correct?
 - ▶ We guarantee the generated program is correct

Pre-requisites

- ▶ Prerequisite: Comp-1000, Comp-2120
 - ▶ Assignments will be implemented in Java.
 - ▶ Need to be very good at debugging Java programs
 - ▶ Better to have Comp 2310.
 - ▶ Better to have Comp 2560



Blue colour: General

Orange colour: Honours

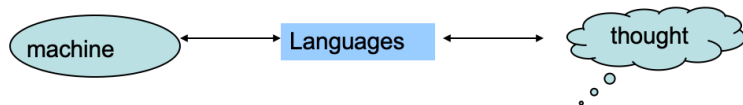


Course description

- ▶ Course title: Computer languages, grammars, and translators
 - ▶ You can interpret this as a “Compiler” course
- ▶ Objective
 - ▶ Knowledge of computer languages and grammars
 - ▶ Able to analyze programs written in various languages
 - ▶ Able to translate languages
- ▶ Contents
 - ▶ Regular expressions, finite automata and language recognizers;
 - ▶ Context free grammar;
 - ▶ Languages parsers.
- ▶ Software tools used
 - ▶ Programming language: Java (including tokenizer, regular expression package)
 - ▶ Lexical analyzer: JLex,
 - ▶ Parser generator: JavaCup

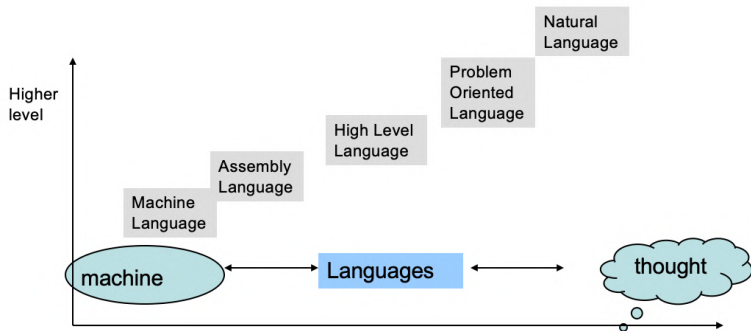
Computer **languages**, grammars, and translators

- ▶ (Human) Language: “a body of words, sounds, and the systems for their use common to a people of the same community”
- ▶ (Formal) Language: “any system of formalized symbols, signs, etc., used or conceived as a means of communication.”
 - ▶ Communicate: to transmit or exchange thought or knowledge.
- ▶ Programming language: communicate between a person and a machine
- ▶ Programming language is an intermediary



Computer **languages**, grammars, and translators

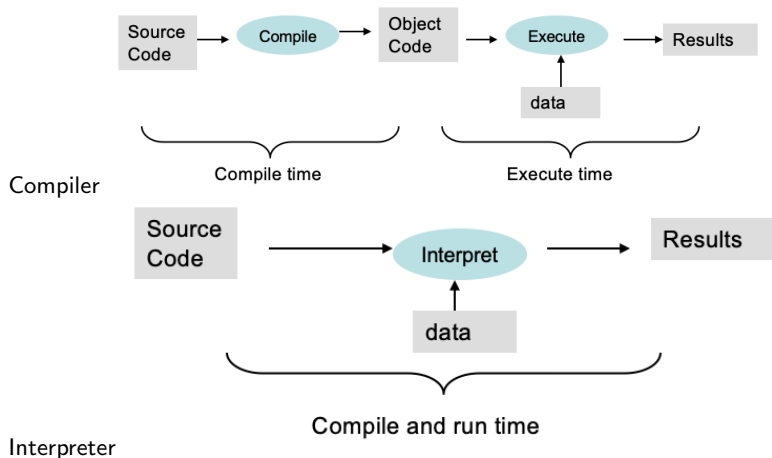
- ▶ Machine language;
- ▶ Assembly language: mnemonic version of machine code;
- ▶ High level language: Java, C#, Pascal;
- ▶ Problem oriented;
- ▶ Natural language.



Computer languages, grammars, and **translators**

- ▶ **Translator:** Translate one language into another language (e.g., from C++ to Java)
 - ▶ A generic term.
- ▶ For high level programming languages (such as java, C):
 - ▶ **Compiler:** translate high level programming language code into host machine's assembly code and execute the translated program at run-time.
 - ▶ **Interpreter:** process the source program and data at the same time. No equivalent assembly code is generated.
- ▶ **Assembler:** translate an assembly language to machine code.

Compiler and Interpreter



How does a compiler work

- ▶ A compiler performs its task in the same way how a human approaches the same problem
- ▶ Consider the following sentence:

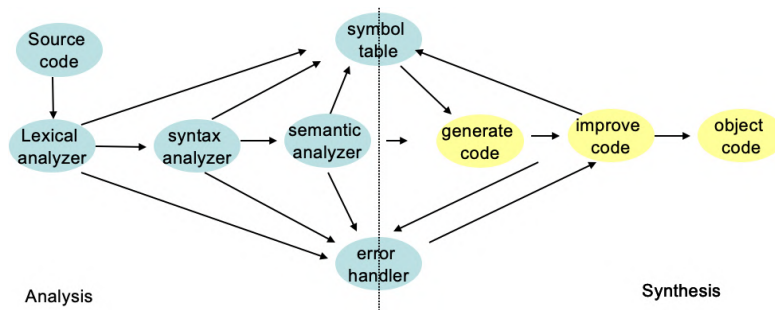
Write a translator

- ▶ We all understand what it means. But how do we arrive at the conclusion?

“Write a translator”

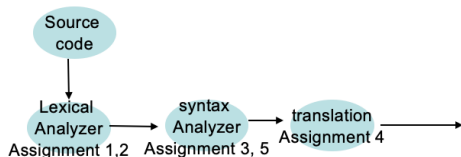
- ▶ Recognize characters (alphabet, mathematical symbols, punctuations).
 - ▶ 16 explicit (alphabets), 2 implicit (blanks)
- ▶ Group characters into logical entities (words).
 - ▶ 3 words.
 - ▶ Lexical analysis
- ▶ Check the words form a structurally correct sentence
 - ▶ “translator a write” is not a correct sentence
 - ▶ Syntactic analysis
- ▶ Check that the combination of words make sense
 - ▶ “dig a translator” is a syntactically correct sentence
 - ▶ Semantic analysis
- ▶ Plan what you have to do to accomplish the task
 - ▶ Code generation
- ▶ Execute it.

The structure (phases) of a compiler



- ▶ Front end (analysis): depend on source language, independent on machine
 - ▶ This is what we will focus (mainly the blue parts).
- ▶ Back end (synthesis): dependent on machine and intermediate code, independent of source code.

Assignments



- ▶ Our focus is the front end
- ▶ Automated generation of lexical analyzer
- ▶ Automated generation of syntax analyzer
- ▶ Assignment 1: analyze without tools
- ▶ Assignment 3: Bottom-up parser
- ▶ Assignment 5: Top-down recursive descent parser.

Assignments

Assignment 1 (warm up): The task is to tokenize a program (4%)

Approach 1 : barehanded without any tools.

Approach 2 : with some help from regular expression

You will see the difficulty to analyse programs without advanced tools such as Jlex and Java Cup.

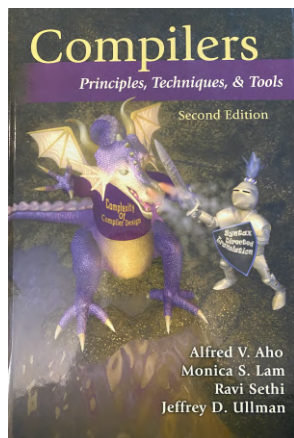
Assignment 2 (4%) Use JLex to build a lexical analyzer for tiny program

Assignment 3 (4%) Use JavaCup to generate a parser

Assignment 4 (4%) Translate the tiny program to Java and actually run it.

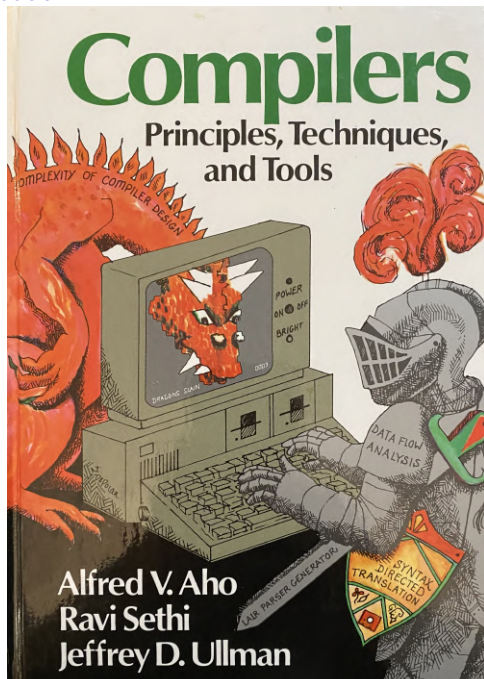
Assignment 5 (4%) Manually write a recursive descent parsing

Textbook, the dragon book



Compilers: Principles, Techniques, and Tools (2nd Edition) by Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman (Aug 31, 2006)

First 5 chapters

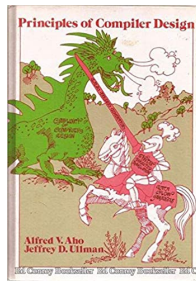


Turing Award Goes to Creators of Computer Programming Building Blocks

Jeffrey Ullman and Alfred Aho developed many of the fundamental concepts that researchers use when they build new software.



Jeffrey Ullman, left, and Alfred Aho were pioneers in creating the “compilers” that translate programming languages into the ones and zeros that computers understand. Stanford University School of Engineering; Eileen Barosso



Programming language experts win 'Nobel Prize of computing', Fortune March 31, 2021

Aho and Ullman published *Principles of Compiler Design*, which also became a classic, teaching generations of students how to create compilers and how to think about the theory of computer languages. The textbook became known as “the Dragon Book” among computer science students because it featured a dragon on its cover, resembling some magical tome Harry Potter might cart around Hogwarts, except the fictional wizard hadn’t been invented yet. “I am often told that putting this funny cover on the Dragon Book attracted students to study computer science,” Ullman says. **“A lot of students liked the idea of walking around campus with this weird book under their arm.”**

Why this course

- ▶ Every university offers this type of courses.
 - ▶ Also called “Compiler” course
- ▶ Skills learnt
 - ▶ write a parser
 - ▶ process programs
 - ▶ re-engineer and migrate programs
 - ▶ Migrate from C++ to C#
 - ▶ ...
- ▶ process data
 - ▶ Xml, web logs, social networks, ...

Why this course is challenging

- ▶ It involves theory
- ▶ Theoretical aspects of programming
- ▶ The science of developing a large program
- ▶ Are you good at Comp-2310?

Why this course is challenging

- ▶ It is about generating programs
 - ▶ It is automated programming, meta-programming
 - ▶ We write “programs” (aka specifications) that can generate programs(aka parser) that can process programs
 - ▶ Are you good at programming in general?
- ▶ It is about 100% correct programming
 - ▶ Have you ever seen a compiler working wrong sometimes?
 - ▶ Your code need to pass our test cases
- ▶ It involves lots of debugging
 - ▶ Debugging the high level specification/rules/grammar
 - ▶ Debugging the generated code
 - ▶ Each assignment may need to try hundreds of times
 - ▶ Are you good at Comp-2120?

(Animation can be viewed using Adobe Acrobat Reader)

Evaluation Scheme

5 Lab Assignments	Midterm 1	Midterm 2	Final Exam	Quizzes	Total
20 %	15%	20%	40%	5	100

Exam dates :

- Midterm exam will be announced later.

Lab assignments due dates : Every other week.

Labs

- ▶ There will be 10 lab sessions. In each odd-numbered lab session (1, 3, 5, 7 and 9), the lab assignment will be explained, and students will start working on it.
- ▶ Lab assignments will be submitted in even-numbered lab sessions (2, 4, 6, 8 and 10).
- ▶ Each lab assignment is worth 4% of the course grade.